

Trabajo Final de Inteligencia Artificial

Tema 2

Construcción de resúmenes como selección óptima de segmentos

Autor: Rubén Martínez Rojas

Grupo C-311

Curso 2025–2026

Índice

1. Introducción	3
2. Descripción del problema	4
2.1. Contexto	4
2.2. Definición del problema	4
2.3. Objetivos	5
3. Modelado formal	5
3.1. Representación de fragmentos	5
3.2. Relevancia	6
3.3. Coherencia	6
3.4. Restricción de duración	6
3.5. Función objetivo	7
4. Diseño de la solución	7
4.1. Arquitectura general	7
4.2. Preparación del dataset	8
4.3. Evaluación mediante LLM	10
4.4. Beam Search	11
4.5. Refinamiento mediante juez macro	12
5. Implementación	13
5.1. Tecnologías utilizadas	13
5.2. Estructura del proyecto	14
5.3. Configuración del LLM	15
5.4. Caché de respuestas	16
6. Metodología experimental	17
6.1. Dataset utilizado	17
6.2. Instancias de prueba	18
6.2.1. Instancias sintéticas y de <i>benchmark</i>	18
6.2.2. Instancias de escalado	19
6.2.3. Instancias reales y mini videos	19
6.3. Configuración experimental	20
6.4. Métricas	21
6.4.1. Métricas estructurales	21
6.4.2. Métricas basadas en LLM	22
6.4.3. Evaluación global del resumen	23
7. Resultados y análisis	23
7.1. Casos sintéticos	23
7.2. Casos benchmark	24
7.2.1. Salto de coherencia (<i>bench_static_vs_dynamic</i>)	25
7.2.2. Fragmento irrelevante (<i>bench_irrelevant_middle</i>)	25
7.2.3. Input permutado (<i>bench_disordered</i>)	25

7.2.4. Síntesis de la suite <i>benchmark</i>	26
7.3. Escalabilidad	26
7.4. Discusión	27
8. Limitaciones y mejoras futuras	28
8.1. Limitaciones	29
8.2. Mejoras futuras	30
9. Conclusiones	30
9.1. ¿Se resolvió el problema?	30
9.2. ¿Qué aportó el uso del LLM?	31
9.3. ¿Qué harías en una versión futura?	31
A. Configuraciones experimentales	31
B. Prompts utilizados	32
B.1. Relevancia de fragmento (<i>fragment_prompt</i>)	32
B.2. Coherencia entre pares (<i>coherence_prompt</i>)	32
B.3. Evaluación global del resumen (<i>summary_evaluation_prompt</i>)	33

1. Introducción

La generación automática de resúmenes constituye una de las aplicaciones más relevantes del procesamiento del lenguaje natural, especialmente en contextos donde es necesario condensar grandes volúmenes de información para facilitar su consulta y comprensión. En el caso de los contenidos audiovisuales educativos, la duración de los materiales y la cantidad de información presentada dificultan la revisión rápida de los temas abordados, lo que hace necesario disponer de mecanismos capaces de producir versiones resumidas que preserven las ideas principales.

El presente trabajo desarrolla una solución para el Tema 2 del Proyecto Final de la asignatura Inteligencia Artificial [1], titulado “Construcción de resúmenes como selección óptima de segmentos”. El problema consiste en seleccionar y ordenar un subconjunto de fragmentos extraídos de un video educativo de manera que el resumen resultante represente adecuadamente el contenido original sin exceder una duración máxima establecida. La calidad del resumen depende tanto de la importancia individual de los fragmentos seleccionados como de la coherencia existente entre ellos cuando se presentan de forma consecutiva.

Desde el punto de vista de la optimización, el problema puede interpretarse como una tarea de selección bajo restricciones. Cada fragmento posee una duración asociada que consume parte del presupuesto temporal disponible, mientras que su incorporación al resumen aporta una determinada utilidad relacionada con su relevancia informativa. Adicionalmente, el orden en que los fragmentos son presentados influye en la calidad global del resumen, por lo que la solución debe considerar simultáneamente la selección de fragmentos y su secuencia de reproducción.

Como parte de los requisitos del proyecto, se incorpora un modelo de lenguaje de gran tamaño (LLM) dentro del sistema. En esta propuesta, el modelo de lenguaje se utiliza para evaluar la relevancia de cada fragmento y la coherencia entre pares de fragmentos. Estas valoraciones son posteriormente empleadas por el algoritmo de optimización para construir candidatos de resumen. De esta forma, el LLM aporta conocimiento semántico sobre el contenido textual mientras que el proceso de búsqueda y optimización permanece a cargo de un algoritmo combinatorio independiente.

La solución desarrollada se basa en una arquitectura compuesta por tres etapas principales. En primer lugar, se realiza un precálculo de puntuaciones de relevancia y coherencia mediante el modelo de lenguaje. Posteriormente, un algoritmo de beam search explora distintas combinaciones de fragmentos respetando la restricción de duración y conservando únicamente las alternativas más prometedoras. Finalmente, los mejores candidatos obtenidos son sometidos a una evaluación global para seleccionar la versión final del resumen.

El objetivo del trabajo es diseñar e implementar un sistema capaz de generar resúmenes coherentes y representativos a partir de fragmentos de videos educativos, integrando técnicas de optimización y modelos de lenguaje dentro de un mismo proceso. Asimismo, se estudia el comportamiento de la solución mediante diferentes instancias de prueba y configuraciones experimentales con el fin de analizar sus fortalezas, limitaciones y posibilidades de mejora.

2. Descripción del problema

2.1. Contexto

Los videos educativos constituyen una fuente importante de información para el aprendizaje en diferentes áreas del conocimiento. La generación de resúmenes extractivos —selección de un subconjunto del contenido original— es un problema clásico en procesamiento del lenguaje natural [4]. Sin embargo, la duración de estos materiales puede dificultar la localización rápida de conceptos específicos o la revisión general de los contenidos tratados. Una alternativa para reducir este problema consiste en generar resúmenes que concentren las ideas más importantes del video y permitan al usuario obtener una visión general de su contenido en un tiempo significativamente menor.

En este trabajo se considera un escenario en el que cada video ha sido previamente segmentado en fragmentos temporales y cada fragmento dispone de una transcripción textual asociada. A partir de estos fragmentos, el sistema debe construir un resumen que represente adecuadamente la información contenida en el material original. Debido a que el resumen debe respetar una duración máxima establecida, no es posible incluir todos los fragmentos disponibles, por lo que resulta necesario decidir cuáles conservar y cuáles descartar.

La generación del resumen no depende únicamente de la importancia individual de cada fragmento. Un resumen compuesto por fragmentos muy relevantes puede resultar difícil de comprender si las transiciones entre ellos son poco naturales o si el orden de presentación rompe la continuidad temática. Por esta razón, además de valorar la información aportada por cada fragmento, es necesario considerar la coherencia existente entre los elementos seleccionados.

2.2. Definición del problema

Sea un conjunto de fragmentos extraídos de un video educativo. Cada fragmento está caracterizado por una transcripción textual, una duración y la información temporal correspondiente a su posición dentro del video original. Además, se dispone de un límite de duración que establece la longitud máxima permitida para el resumen final.

El problema consiste en seleccionar un subconjunto de fragmentos cuya duración total no exceda dicho límite y determinar un orden de reproducción para los fragmentos seleccionados. El objetivo es obtener un resumen que preserve la mayor cantidad posible de información relevante y que mantenga una secuencia coherente entre sus diferentes partes.

Para evaluar la calidad de una solución se consideran dos criterios principales. El primero es la relevancia de los fragmentos individuales, entendida como la contribución informativa de cada uno al contenido global del video. El segundo es la coherencia entre fragmentos consecutivos, entendida como el grado en que dos fragmentos pueden aparecer uno después del otro manteniendo continuidad temática y discursiva. Una solución de alta calidad debe equilibrar ambos criterios mientras satisface la restricción de duración impuesta.

La naturaleza combinatoria del problema surge del elevado número de posibles subconjuntos y ordenaciones que pueden construirse a partir de los fragmentos disponibles. A medida que aumenta la cantidad de fragmentos, el espacio de búsqueda crece rápidamente, lo que dificulta encontrar soluciones de calidad mediante una exploración exhaustiva. Por esta razón resulta necesario emplear técnicas de búsqueda y optimización que permitan explorar únicamente las alternativas más prometedoras.

2.3. Objetivos

El objetivo general del trabajo es desarrollar un sistema capaz de construir resúmenes de videos educativos mediante la selección y ordenación de fragmentos bajo una restricción de duración máxima, incorporando un modelo de lenguaje como parte funcional del proceso de evaluación.

Para alcanzar este objetivo se propone representar formalmente el problema como una tarea de optimización en la que cada solución corresponde a una secuencia ordenada de fragmentos. Asimismo, se busca utilizar un modelo de lenguaje para estimar la relevancia de los fragmentos y la coherencia entre pares de fragmentos, aprovechando su capacidad para analizar información textual y capturar relaciones semánticas que resultan difíciles de modelar mediante reglas manuales.

Además, se pretende diseñar e implementar un algoritmo de búsqueda capaz de generar resúmenes de calidad sin necesidad de explorar exhaustivamente todas las combinaciones posibles. Finalmente, se realizará una evaluación experimental sobre diferentes instancias de prueba con el propósito de analizar el comportamiento del sistema, estudiar la influencia del modelo de lenguaje en el proceso de selección y determinar las principales limitaciones de la propuesta desarrollada.

3. Modelado formal

3.1. Representación de fragmentos

La entrada del problema está compuesta por un conjunto finito de fragmentos obtenidos a partir de la transcripción de un video educativo. Cada fragmento contiene el texto correspondiente a una porción del video, así como información temporal asociada a su posición dentro del material original.

Formalmente, cada fragmento puede representarse mediante una tupla:

$$f_i = (t_i, d_i, s_i, e_i)$$

donde t_i representa el texto del fragmento, d_i su duración en segundos, s_i el instante de inicio y e_i el instante de finalización dentro del video original. Sea

$$F = \{f_1, f_2, \dots, f_n\}$$

el conjunto de todos los fragmentos disponibles. Además, se dispone de una duración máxima permitida para el resumen, denotada por D_{max} .

Una solución del problema consiste en construir una secuencia ordenada de fragmentos seleccionados:

$$S = (f_{i_1}, f_{i_2}, \dots, f_{i_k})$$

donde cada fragmento puede aparecer como máximo una vez.

3.2. Relevancia

Con el objetivo de estimar la importancia informativa de cada fragmento, se asocia una puntuación de relevancia a cada elemento del conjunto de entrada.

Sea

$$r_i \in [0, 1]$$

la relevancia asignada al fragmento (f_i) . Valores cercanos a 1 indican que el fragmento contiene información especialmente útil para la construcción de un resumen educativo, mientras que valores cercanos a 0 representan fragmentos con escasa contribución al contenido principal.

Estas puntuaciones son obtenidas mediante un modelo de lenguaje, el cual analiza el texto de cada fragmento de manera independiente y devuelve un valor numérico normalizado entre 0 y 1.

3.3. Coherencia

Además de la relevancia individual de los fragmentos, es necesario considerar la relación existente entre fragmentos consecutivos dentro del resumen.

Para cada par ordenado de fragmentos $((f_i, f_j))$ se define una puntuación de coherencia:

$$c_{ij} \in [0, 1]$$

que representa el grado de continuidad temática y discursiva existente al colocar (f_j) inmediatamente después de (f_i) .

Valores elevados de (c_{ij}) indican que ambos fragmentos forman una transición natural dentro del resumen, mientras que valores bajos sugieren cambios bruscos de tema o secuencias poco comprensibles para el lector.

Al igual que ocurre con la relevancia, estas puntuaciones son calculadas mediante un modelo de lenguaje a partir del contenido textual de ambos fragmentos.

3.4. Restricción de duración

La duración total del resumen no puede exceder el límite establecido para la instancia.

Si la solución seleccionada contiene los fragmentos

$$S = (f_{i_1}, f_{i_2}, \dots, f_{i_k}),$$

entonces debe cumplirse que

$$\sum_{j=1}^k d_{i_j} \leq D_{max}.$$

Esta restricción garantiza que el resumen generado respete la longitud máxima especificada para el problema.

3.5. Función objetivo

El objetivo consiste en encontrar una secuencia de fragmentos que maximice simultáneamente la relevancia del contenido seleccionado y la coherencia entre fragmentos consecutivos.

Para una solución

$$S = (f_{i_1}, f_{i_2}, \dots, f_{i_k}),$$

la calidad puede expresarse mediante una combinación de ambos criterios:

$$\text{Score}(S) = \sum_{j=1}^k r_{i_j} + \lambda \sum_{j=1}^{k-1} c_{i_j, i_{j+1}} \quad (1)$$

donde (λ) representa el peso asignado a la coherencia dentro de la evaluación total.

En la implementación desarrollada, la búsqueda utiliza un valor de $(\lambda = 0.25)$, lo que permite priorizar la selección de fragmentos relevantes sin ignorar la calidad de las transiciones entre ellos.

Por tanto, el problema consiste en encontrar la secuencia válida de fragmentos que maximice la función objetivo anterior y satisfaga simultáneamente la restricción de duración establecida para el resumen.

4. Diseño de la solución

4.1. Arquitectura general

La solución desarrollada sigue una arquitectura modular compuesta por tres etapas principales: evaluación semántica mediante un modelo de lenguaje, búsqueda combinatoria de candidatos y evaluación global de los resúmenes generados. Esta separación permite aprovechar las capacidades de comprensión textual de los modelos de lenguaje sin delegar en ellos el proceso de optimización, que permanece completamente controlado por algoritmos deterministas implementados en el sistema.

El flujo de ejecución comienza con la carga de una instancia del problema. Cada instancia contiene un conjunto de fragmentos obtenidos a partir de la transcripción de un video educativo y una duración máxima permitida para el resumen final. Una vez cargada la información, se inicia una fase de precálculo en la que se evalúan todos los fragmentos disponibles mediante el modelo de lenguaje.

Durante esta etapa se obtiene una puntuación de relevancia para cada fragmento y una matriz de coherencia entre todos los pares posibles de fragmentos. La relevancia representa la utilidad informativa de un fragmento dentro de un resumen educativo, mientras que la coherencia mide la calidad de la transición producida al situar un fragmento inmediatamente después de otro. Estas puntuaciones son almacenadas en una estructura de datos que posteriormente será utilizada por el algoritmo de búsqueda.

Una vez completado el precálculo, el sistema ejecuta un algoritmo de beam search encargado de construir soluciones candidatas. A diferencia de otros enfoques en los que el modelo de lenguaje participa directamente durante la exploración, en esta propuesta todas las consultas al LLM se realizan antes de comenzar la búsqueda. De esta forma, el algoritmo puede explorar múltiples combinaciones de fragmentos utilizando únicamente las puntuaciones previamente calculadas, reduciendo significativamente el número de llamadas a la API y mejorando la eficiencia del proceso.

El beam search genera secuencias ordenadas de fragmentos que respetan la restricción de duración máxima y asigna una puntuación local a cada candidato en función de la relevancia acumulada y de la coherencia entre fragmentos consecutivos. Durante la exploración se conservan únicamente las alternativas más prometedoras, limitando así el crecimiento del espacio de búsqueda.

En lugar de seleccionar directamente el candidato con mejor puntuación local, el sistema conserva los mejores resultados obtenidos por el beam search y realiza una etapa adicional de refinamiento. En esta fase, el texto completo de cada resumen candidato es evaluado nuevamente mediante el modelo de lenguaje, que proporciona una valoración global basada en la relevancia del contenido seleccionado y en la coherencia de la secuencia resultante.

Finalmente, el resumen con mayor evaluación global es devuelto como solución del problema. El resultado consiste en una secuencia ordenada de fragmentos cuya duración total no supera el límite establecido y que busca maximizar simultáneamente la representatividad del contenido y la coherencia narrativa del resumen generado.

La Figura 1 resume el flujo general de funcionamiento del sistema.

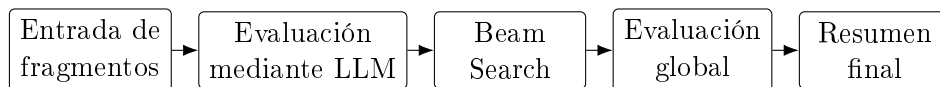


Figura 1: Flujo general del sistema: precálculo semántico, búsqueda combinatoria y refinamiento global.

4.2. Preparación del dataset

El sistema desarrollado trabaja sobre instancias construidas a partir de transcripciones de videos educativos. Con el fin de disponer de datos adecuados para la experimentación, se implementó un proceso de preparación que transforma los archivos de subtítulos originales en instancias estructuradas del problema de optimización.

La generación de las instancias reales se realiza mediante el script `prepare_dataset.py`. Como entrada, el programa recibe archivos de subtítulos en

formato SRT correspondientes a diferentes videos educativos. Cada bloque de subtítulos contiene un intervalo temporal y el texto pronunciado durante dicho intervalo. El sistema procesa estos archivos eliminando marcas no informativas y convirtiendo los tiempos a segundos para facilitar su tratamiento posterior.

Una vez cargados los subtítulos, los bloques individuales son agrupados en fragmentos de mayor tamaño mediante una estrategia de agregación temporal. El objetivo es obtener segmentos suficientemente largos para contener información con significado semántico completo, evitando al mismo tiempo fragmentos excesivamente extensos. En la implementación utilizada, los subtítulos consecutivos se agrupan hasta alcanzar aproximadamente entre veinte y veinticinco segundos de duración acumulada.

Cada fragmento generado se representa mediante una estructura que almacena el texto concatenado, la duración del segmento y su posición temporal dentro del video original. Formalmente, cada instancia contiene una colección ordenada de fragmentos junto con la duración máxima permitida para el resumen.

Para determinar dicha restricción se calcula primero la duración total del material de entrada:

$$D_{total} = \sum_{i=1}^n d_i,$$

donde d_i representa la duración del fragmento f_i . A continuación, la duración máxima del resumen se fija como un veinticinco por ciento de la duración total disponible:

$$D_{max} = 0,25 \cdot D_{total}.$$

Esta elección permite generar resúmenes significativamente más breves que el contenido original manteniendo un margen suficiente para incluir los fragmentos más representativos.

Las instancias resultantes se almacenan en formato JSON dentro del directorio `data/instances/`. Cada archivo contiene la lista de fragmentos generados y la restricción temporal asociada. Los videos reales procesados producen instancias denominadas `video_1.json`, `video_2.json`, `video_3.json` y `video_4.json`, con tamaños comprendidos aproximadamente entre veinticinco y cincuenta fragmentos.

Además de las instancias completas, se generaron conjuntos derivados destinados a facilitar la experimentación. Por una parte, el script `prepare_mini_instances.py` permite construir versiones reducidas de los videos conservando únicamente los primeros fragmentos consecutivos. Estas instancias, denominadas `mini_video_*`, mantienen la estructura del problema original pero reducen considerablemente el coste computacional asociado a la evaluación mediante modelos de lenguaje.

Por otra parte, se construyeron instancias de escalado basadas en diferentes cantidades de contenido procedente de un mismo video. Estas instancias, identificadas como `dur_5min`, `dur_10min`, `dur_15min` y `dur_20min`, permiten analizar cómo evoluciona el comportamiento del sistema a medida que aumenta el número de fragmentos disponibles y, en consecuencia, el tamaño del espacio de búsqueda.

Finalmente, para complementar los datos reales, se diseñaron varias instancias sintéticas de pequeño tamaño. Estas instancias contienen únicamente cinco fragmentos y fueron construidas manualmente con el objetivo de reproducir situaciones específicas de interés experimental, como la presencia de fragmentos irrelevantes o la existencia de diferentes alternativas de ordenación. Su reducido tamaño permite comparar de forma controlada el comportamiento de los distintos solvers y facilita la interpretación de los resultados obtenidos.

La combinación de instancias reales, instancias derivadas e instancias sintéticas proporciona un conjunto de pruebas suficientemente variado para estudiar tanto la calidad de los resúmenes generados como las limitaciones computacionales de la propuesta desarrollada.

4.3. Evaluación mediante LLM

El modelo de lenguaje constituye el componente encargado de proporcionar las valoraciones semánticas utilizadas posteriormente por el algoritmo de optimización. Su función no consiste en generar directamente el resumen, sino en estimar la calidad informativa de los fragmentos y la coherencia existente entre ellos.

La evaluación se realiza en una fase de precálculo previa a la búsqueda combinatoria. De esta forma, todas las llamadas al modelo de lenguaje se ejecutan una única vez para cada instancia, evitando consultas repetidas durante la exploración del espacio de soluciones.

El sistema calcula dos tipos de puntuaciones.

En primer lugar, para cada fragmento individual se obtiene una puntuación de relevancia. Para ello se construye un prompt que solicita al modelo valorar la utilidad del fragmento dentro de un resumen educativo y devolver exclusivamente un número comprendido entre 0 y 1. Cuanto mayor sea la puntuación obtenida, mayor será la importancia atribuida al fragmento dentro del contenido global del video.

En segundo lugar, se calcula una puntuación de coherencia para cada par ordenado de fragmentos. En este caso el modelo recibe los textos de ambos fragmentos y debe estimar el grado de continuidad temática y discursiva que existe al colocar el segundo inmediatamente después del primero. Al igual que en la evaluación de relevancia, la respuesta esperada es un valor numérico normalizado entre 0 y 1.

Las puntuaciones obtenidas se almacenan en dos estructuras de datos. La primera es un vector de relevancias:

$$R = (r_1, r_2, \dots, r_n),$$

donde (r_i) representa la relevancia del fragmento (f_i) .

La segunda es una matriz de coherencia:

$$C = \begin{bmatrix} 0 & c_{12} & \cdots & c_{1n} \\ c_{21} & 0 & \cdots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & \cdots & 0 \end{bmatrix}$$

donde cada elemento (c_{ij}) expresa la coherencia obtenida al situar el fragmento (f_j) después del fragmento (f_i) .

Con el objetivo de reducir el número de llamadas a la API y acelerar los experimentos, el sistema incorpora un mecanismo de caché persistente. Cada prompt enviado al modelo se almacena junto con su respuesta, de manera que futuras ejecuciones puedan reutilizar automáticamente los resultados ya calculados.

La implementación permite utilizar diferentes proveedores de modelos de lenguaje mediante una interfaz común. Actualmente se soportan Google Gemini y APIs compatibles con OpenAI, incluyendo Groq. Esta abstracción facilita la comparación entre modelos sin necesidad de modificar el resto del sistema.

El resultado final de esta etapa es una matriz completa de puntuaciones de relevancia y coherencia que será utilizada posteriormente por el algoritmo de beam search para construir los resúmenes candidatos.

4.4. Beam Search

Una vez calculadas las puntuaciones de relevancia y coherencia mediante el modelo de lenguaje, el sistema debe determinar qué fragmentos incluir en el resumen final. Dado que el número de posibles combinaciones crece rápidamente con la cantidad de fragmentos disponibles, una búsqueda exhaustiva resulta poco práctica incluso para instancias de tamaño moderado. Por esta razón se emplea un algoritmo de *beam search* [5], capaz de explorar únicamente las alternativas más prometedoras del espacio de búsqueda.

El objetivo del algoritmo es construir progresivamente secuencias ordenadas de fragmentos que respeten la restricción de duración máxima y maximicen la función objetivo definida en la Sección 3. Cada estado de búsqueda representa una solución parcial formada por una secuencia de fragmentos ya seleccionados, la puntuación acumulada obtenida hasta el momento y la duración total consumida.

Inicialmente el algoritmo parte de una solución vacía. En cada iteración se generan nuevas alternativas añadiendo un fragmento no utilizado a cada una de las secuencias parciales presentes en el beam. Una expansión únicamente se considera válida cuando la duración total resultante no supera el límite establecido para la instancia.

La puntuación de una solución parcial se calcula de forma incremental. Cuando se añade un nuevo fragmento (f_j), la puntuación aumenta en función de su relevancia individual y, si existe un fragmento anterior en la secuencia, también incorpora la contribución de la coherencia entre ambos elementos:

$$\Delta = r_j + \lambda, c_{ij},$$

donde (f_i) representa el último fragmento de la secuencia actual y (λ) es el peso asignado a la coherencia.

Tras generar todas las expansiones posibles, los candidatos se ordenan según su puntuación acumulada y únicamente se conservan los mejores. Este conjunto reducido constituye el nuevo beam que será expandido en la siguiente iteración. El parámetro que controla el número máximo de candidatos conservados recibe el nombre de *beam width*.

La principal ventaja de este enfoque es que permite explorar múltiples alternativas simultáneamente sin necesidad de mantener todas las combinaciones posibles.

Mientras que una búsqueda exhaustiva tendría un crecimiento exponencial, el beam search limita deliberadamente el número de estados activos, logrando tiempos de ejecución mucho más reducidos a costa de renunciar a la garantía de optimalidad global.

Además de devolver el mejor candidato encontrado, la implementación desarrollada permite conservar varios resúmenes de alta calidad. Para ello se mantiene una lista de los (M) candidatos con mayor puntuación local obtenidos durante la exploración. Esta característica resulta especialmente útil porque posibilita una fase posterior de refinamiento mediante un juez global basado en LLM.

En la configuración utilizada durante los experimentos se emplea un ancho de beam de diez candidatos (*beam width* = 10). Este valor proporciona un equilibrio adecuado entre calidad de las soluciones y coste computacional, permitiendo explorar diferentes alternativas sin que el tiempo de ejecución aumente de forma significativa.

El resultado de esta etapa es un conjunto reducido de resúmenes candidatos que cumplen la restricción de duración y presentan una combinación favorable de relevancia y coherencia. Estos candidatos son posteriormente evaluados mediante un análisis global para seleccionar la versión final del resumen.

4.5. Refinamiento mediante juez macro

Aunque las puntuaciones de relevancia y coherencia calculadas durante la fase de precálculo permiten estimar la calidad de una solución, estas valoraciones se obtienen a partir de fragmentos individuales o pares de fragmentos. Como consecuencia, la función objetivo utilizada durante el beam search proporciona una medida local de calidad que no siempre refleja adecuadamente el comportamiento global del resumen completo.

Con el fin de incorporar una evaluación a nivel de resumen, la solución desarrollada incluye una etapa adicional de refinamiento basada en un modelo de lenguaje. Esta fase se ejecuta una vez finalizado el beam search y utiliza el propio LLM como juez global de los candidatos generados.

En lugar de considerar únicamente la mejor solución obtenida por la búsqueda, el sistema conserva los (M) candidatos con mayor puntuación local. Cada uno de estos candidatos se transforma en un texto continuo mediante la concatenación de los fragmentos seleccionados siguiendo el orden determinado durante la búsqueda. El resultado es un conjunto reducido de resúmenes completos que serán analizados individualmente.

Para cada resumen candidato se construye un prompt específico que solicita al modelo de lenguaje una evaluación global del contenido. El modelo debe analizar el resumen completo teniendo en cuenta la representatividad de la información seleccionada, la coherencia de la secuencia de fragmentos y la calidad general del resultado obtenido. Como respuesta, el sistema solicita un objeto JSON con tres puntuaciones normalizadas entre 0 y 1:

- **Relevance:** medida de la relevancia global del contenido seleccionado.
- **Coherence:** valoración de la continuidad y coherencia entre los fragmentos del resumen.

- **Overall:** evaluación global del resumen completo.

Estas puntuaciones se encapsulan en una estructura de datos denominada `SummaryEvaluation`, que almacena las tres valoraciones devueltas por el modelo.

Una vez evaluados todos los candidatos, el sistema selecciona como solución final aquel resumen que obtiene la mayor puntuación *overall*. De esta forma, la decisión final no depende exclusivamente de la función objetivo utilizada durante el beam search, sino también de una valoración semántica global realizada sobre el resumen completo.

Este mecanismo puede interpretarse como una estrategia de reordenamiento o *reranking* de candidatos. El beam search actúa como un generador eficiente de posibles soluciones, mientras que el juez macro introduce una segunda etapa de selección basada en una comprensión más amplia del contenido textual. Gracias a esta combinación, el sistema puede corregir situaciones en las que una solución presenta una puntuación local elevada pero genera un resumen menos satisfactorio cuando se analiza de forma conjunta.

La implementación permite controlar el número de candidatos sometidos a esta evaluación mediante el parámetro `summary_refine_top_m`. Cuando este valor es mayor que cero, se activa automáticamente la fase de refinamiento. En los experimentos realizados se emplea un valor por defecto de tres candidatos, lo que proporciona un equilibrio razonable entre calidad de la selección final y coste computacional adicional.

El resultado de esta etapa es el resumen definitivo devuelto por el sistema, acompañado de las puntuaciones globales estimadas por el modelo de lenguaje para facilitar su posterior análisis experimental.

5. Implementación

5.1. Tecnologías utilizadas

La implementación del sistema se realizó íntegramente en Python, aprovechando su amplio ecosistema de bibliotecas para procesamiento de datos, interacción con modelos de lenguaje y desarrollo de algoritmos de optimización. La elección de este lenguaje permite combinar de forma sencilla componentes de inteligencia artificial con estructuras de búsqueda y evaluación experimental.

El núcleo del sistema se encuentra organizado en módulos independientes encargados de la representación del problema, la evaluación mediante modelos de lenguaje, la búsqueda de soluciones y la ejecución de experimentos. Esta estructura modular facilita el mantenimiento del código y permite sustituir componentes concretos sin afectar al resto de la aplicación.

Para la integración con modelos de lenguaje se utilizaron dos familias de APIs. La primera corresponde a Google Gemini mediante la biblioteca oficial `google-generativeai`. La segunda utiliza la biblioteca `OpenAI`, que proporciona compatibilidad tanto con los modelos de OpenAI como con otros proveedores que implementan la misma interfaz, entre ellos Groq.

La gestión de credenciales y parámetros de configuración se realiza mediante variables de entorno cargadas con la biblioteca `python-dotenv`. Este mecanismo evita almacenar claves de acceso directamente en el código fuente y permite cambiar fácilmente de proveedor o modelo durante la ejecución de experimentos.

Las instancias del problema se almacenan utilizando el formato JSON, que ofrece una representación sencilla y fácilmente intercambiable de los fragmentos, sus transcripciones y las restricciones de duración asociadas. Asimismo, los resultados experimentales se exportan en formato CSV para facilitar su posterior análisis y comparación.

Por último, la implementación incorpora un sistema de caché persistente basado en archivos JSON. Este mecanismo permite reutilizar respuestas obtenidas previamente del modelo de lenguaje, reduciendo el número de llamadas a la API y acelerando significativamente la ejecución de experimentos repetidos.

En conjunto, estas tecnologías proporcionan una infraestructura flexible que permite evaluar diferentes modelos de lenguaje, ejecutar algoritmos de búsqueda combinatoria y realizar análisis experimentales de manera reproducible.

5.2. Estructura del proyecto

La implementación se organizó siguiendo una estructura modular con el objetivo de separar claramente las distintas responsabilidades del sistema. Esta organización facilita el mantenimiento del código, la reutilización de componentes y la incorporación de nuevas funcionalidades sin modificar el resto de la arquitectura.

El proyecto se encuentra dividido en varios módulos principales, cada uno encargado de una fase específica del proceso de construcción de resúmenes.

El módulo `llm` contiene toda la funcionalidad relacionada con la interacción con modelos de lenguaje. Dentro de este módulo se implementan los prompts utilizados para evaluar fragmentos y resúmenes, la gestión de llamadas a las distintas APIs compatibles y un sistema de caché destinado a evitar consultas repetidas durante los experimentos.

El módulo `solver` implementa los algoritmos de optimización. En esta parte se encuentra el beam search unificado encargado de construir candidatos de resumen utilizando las puntuaciones de relevancia y coherencia previamente calculadas. También incluye la lógica de refinamiento basada en la evaluación global de los candidatos generados.

El archivo `problem.py` contiene la definición formal de las estructuras de datos principales empleadas por el sistema. En particular, define las clases `Fragment` y `SelectionProblem`, utilizadas para representar los fragmentos de entrada y las instancias completas del problema.

El módulo `experiments` reúne las herramientas necesarias para la ejecución y evaluación de experimentos. Entre sus funciones se incluyen el cálculo de métricas estructurales, la comparación entre diferentes configuraciones del sistema y la generación de tablas de resultados.

Por su parte, el script `run_experiments.py` actúa como punto de entrada principal para la ejecución de pruebas experimentales. Este programa permite seleccionar conjuntos de instancias, configurar parámetros del beam search, activar o desactivar

el uso del modelo de lenguaje y almacenar los resultados obtenidos en archivos CSV para su posterior análisis.

La Figura 2 muestra de forma esquemática la organización general del proyecto.

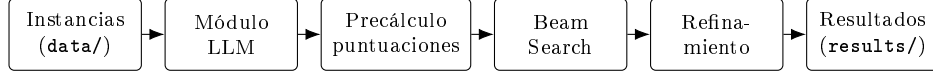


Figura 2: Organización modular del proyecto y flujo de datos entre componentes.

Esta estructura modular permitió desarrollar y evaluar cada componente de manera independiente, simplificando tanto las tareas de depuración como la comparación entre diferentes versiones del sistema.

5.3. Configuración del LLM

La componente basada en modelos de lenguaje se implementa mediante una capa de abstracción que permite utilizar distintos proveedores de manera transparente para el resto del sistema. Esta decisión facilita la experimentación con diferentes modelos sin necesidad de modificar el algoritmo de búsqueda ni el resto de los módulos encargados de la optimización.

La configuración del modelo se centraliza en la clase `LLMClient`, responsable de gestionar la comunicación con los servicios externos, construir las consultas y procesar las respuestas obtenidas. Durante la inicialización, el cliente determina automáticamente qué proveedor debe utilizarse a partir de los parámetros recibidos o de las variables definidas en el archivo de configuración del proyecto.

La implementación soporta actualmente tres modos de funcionamiento. El primero utiliza Google Gemini mediante la biblioteca oficial `google-generativeai`. El segundo emplea la API de OpenAI. El tercero permite utilizar Groq [2], aprovechando su compatibilidad con la interfaz de OpenAI. En los experimentos de este informe se empleó el modelo `llama-3.3-70b-versatile` [3]. Esta arquitectura permite sustituir un proveedor por otro sin alterar el resto del sistema.

La selección del proveedor y del modelo se realiza mediante variables de entorno definidas en el archivo `.env`. Entre los parámetros configurables se encuentran el proveedor utilizado, el nombre del modelo y las credenciales necesarias para acceder al servicio correspondiente. Esta aproximación evita almacenar claves de acceso directamente en el código fuente y facilita la reproducción de los experimentos en distintos entornos de ejecución.

Cuando se utiliza Gemini, el sistema configura automáticamente la clave de acceso mediante la variable `GEMINI_API_KEY`. Por defecto se emplea el modelo `gemini-2.5-flash`, aunque cualquier modelo compatible puede especificarse a través de la configuración. En el caso de OpenAI o Groq, las llamadas se realizan mediante el cliente oficial de OpenAI, utilizando la URL base y la clave de autenticación definidas por el usuario.

Todas las consultas realizadas durante los experimentos utilizan una temperatura igual a cero. Esta configuración reduce la variabilidad de las respuestas generadas y favorece la reproducibilidad de las puntuaciones obtenidas para relevancia y coherencia. Dado que estas puntuaciones se utilizan posteriormente como entrada del

algoritmo de optimización, resulta deseable minimizar el componente aleatorio asociado a la generación de texto.

La implementación incorpora además mecanismos básicos de tolerancia a fallos. Cuando se produce un error temporal o una limitación de cuota por parte del proveedor, el sistema realiza varios intentos antes de abandonar la consulta. En particular, se gestionan explícitamente los errores de tipo *rate limit*, introduciendo pausas de espera antes de volver a enviar la petición. Esta estrategia permite completar experimentos largos de manera más robusta cuando el número de consultas al modelo es elevado.

Con el fin de reducir la carga sobre las APIs externas, todas las respuestas obtenidas son almacenadas localmente mediante un sistema de caché persistente. De esta forma, si una consulta ya ha sido realizada anteriormente, el sistema recupera directamente la respuesta almacenada sin necesidad de generar una nueva llamada al modelo. Esta optimización resulta especialmente importante durante el desarrollo y la repetición de experimentos, ya que disminuye significativamente tanto el tiempo de ejecución como el consumo de recursos asociados al uso del LLM.

Gracias a esta capa de abstracción, el resto de los componentes del sistema opera de forma completamente independiente del proveedor utilizado. El algoritmo de búsqueda únicamente consume las puntuaciones generadas por el cliente LLM, mientras que todos los detalles relacionados con autenticación, comunicación y procesamiento de respuestas permanecen encapsulados dentro de esta interfaz.

5.4. Caché de respuestas

La evaluación mediante modelos de lenguaje constituye la parte más costosa del sistema desde el punto de vista temporal y computacional. Durante la ejecución de los experimentos es frecuente que una misma consulta sea realizada múltiples veces, especialmente cuando se repiten pruebas sobre las mismas instancias o se modifican parámetros del algoritmo de búsqueda. Para evitar llamadas redundantes a la API y reducir significativamente el tiempo de ejecución, se incorporó un mecanismo de almacenamiento local de respuestas.

La implementación utiliza una caché persistente basada en archivos JSON. Cada consulta enviada al modelo de lenguaje se emplea como clave de acceso, mientras que la respuesta generada por el modelo se almacena como valor asociado. De esta forma, cuando el sistema necesita realizar una evaluación, primero verifica si existe una respuesta previamente almacenada para el mismo prompt. Si la entrada ya está disponible en la caché, el resultado se recupera directamente desde el almacenamiento local sin necesidad de contactar nuevamente con el proveedor del modelo.

El mecanismo de caché se encuentra encapsulado en la clase `LLMCache`, ubicada dentro del módulo `src/llm/cache.py`. Durante la inicialización del sistema, esta clase intenta cargar el contenido de un archivo JSON persistente. Si el archivo no existe o contiene información inválida, se crea automáticamente una estructura vacía para comenzar el almacenamiento de nuevas respuestas.

Cuando se produce una consulta al modelo, el cliente LLM verifica primero la existencia de una entrada correspondiente en la caché. En caso de encontrar una coincidencia, la respuesta almacenada se devuelve inmediatamente. Si no existe una

respuesta previa, el sistema realiza la llamada al proveedor configurado, obtiene el resultado y lo incorpora automáticamente a la caché antes de devolverlo al resto de componentes.

Este enfoque ofrece varias ventajas prácticas durante el desarrollo y la experimentación. En primer lugar, reduce considerablemente el número total de llamadas realizadas a las APIs externas, disminuyendo tanto los tiempos de ejecución como el consumo de cuota asociado a los servicios de inferencia. En segundo lugar, mejora la reproducibilidad de los experimentos, ya que una vez almacenada una respuesta, las ejecuciones posteriores utilizarán exactamente los mismos valores sin depender de posibles variaciones producidas por nuevas consultas al modelo.

La caché se utiliza de forma transparente para todas las evaluaciones realizadas por el sistema. Esto incluye las puntuaciones de relevancia asignadas a fragmentos individuales, las evaluaciones de coherencia entre pares de fragmentos y las valoraciones globales efectuadas durante la fase de refinamiento de candidatos. Como consecuencia, los experimentos repetidos sobre las mismas instancias pueden ejecutarse con una velocidad significativamente superior a la de la primera ejecución.

Gracias a esta estrategia, la integración de modelos de lenguaje resulta mucho más eficiente y viable para la realización de múltiples experimentos, manteniendo al mismo tiempo la simplicidad de la arquitectura general del sistema.

6. Metodología experimental

6.1. Dataset utilizado

La evaluación experimental del sistema se apoya en un conjunto de instancias almacenadas en formato JSON dentro del directorio `data/instances/`. Cada instancia representa un problema de selección y ordenación de fragmentos con transcripción textual, duración por segmento, marcas temporales de origen y una restricción `max_duration` que fija la longitud máxima permitida para el resumen.

El procedimiento de construcción de estas instancias a partir de subtítulos en formato SRT, la agregación temporal de bloques en segmentos de aproximadamente veinte a veinticinco segundos y la regla $D_{max} = 0,25 \cdot D_{total}$ se describen con detalle en la Sección 4.2. En la presente sección se resume únicamente la composición del corpus y el alcance de los experimentos ejecutados, con un enfoque orientado a la reproducibilidad de la evaluación.

El corpus se organiza en cuatro familias complementarias. En primer lugar, cuatro instancias completas (`video_1.json` a `video_4.json`) procedentes de videos educativos reales, con entre veinticinco y cincuenta fragmentos y duraciones de origen comprendidas aproximadamente entre diez y veintidós minutos. En segundo lugar, cuatro versiones reducidas (`mini_video_1.json` a `mini_video_4.json`) obtenidas conservando únicamente los diez primeros fragmentos consecutivos de cada video, con el fin de reducir el coste de las consultas al modelo de lenguaje. En tercer lugar, cuatro instancias de escalado (`dur_5min.json` a `dur_20min.json`) extraídas del mismo material que `video_2.json`, pero limitadas progresivamente a aproximadamente cinco, diez, quince y veinte minutos de contenido acumulado. Por último, cinco instancias sintéticas o de *benchmark* de cinco fragmentos, construidas manual-

mente para reproducir escenarios controlados de selección, recorte temporal, ruido semántico y reordenación del input.

Todas las instancias comparten el mismo esquema de datos: lista ordenada de fragmentos con campos `text`, `duration`, `start_time` y `end_time`, junto con el límite temporal del resumen. Esta homogeneidad permite ejecutar el mismo pipeline experimental —precálculo de puntuaciones, beam search y evaluación de métricas— sobre conjuntos de distinto tamaño y procedencia.

Los experimentos cuantitativos documentados en este informe se ejecutaron con el proveedor Groq y el modelo `llama-3.3-70b-versatile`, empleando temperatura cero para favorecer la reproducibilidad de las puntuaciones (véase la Sección 5.3). La suite principal de comparación entre `baseline_beam` y `llm_beam` abarcó las cinco instancias sintéticas y de *benchmark*. Adicionalmente, se evaluó una instancia de escalado (`dur_5min.json`, doce fragmentos) para analizar el comportamiento del sistema fuera del régimen de cinco fragmentos. Las instancias completas `video_*` y las versiones `mini_video_*` permanecen disponibles como material de prueba, pero no formaron parte de la suite cuantitativa principal debido al elevado número de consultas al modelo de lenguaje que requeriría la matriz de coherencia en instancias de mayor tamaño.

6.2. Instancias de prueba

La Tabla 1 resume las instancias disponibles, agrupadas según su procedencia y finalidad experimental. Para cada grupo se indica el número de archivos, el rango de fragmentos por instancia, la duración aproximada del material de origen y el papel que desempeña en la evaluación del sistema.

Cuadro 1: Resumen de instancias de prueba disponibles.

Grupo	Archivos	Fragmentos	Finalidad
Videos reales	<code>video_1–4</code>	25–50	Corpus principal de subtítulos reales
Mini videos	<code>mini_video_1–4</code>	10	Recortes para pruebas de menor coste
Escalado	<code>dur_5min–dur_20min</code>	12–47	Crecimiento de n y del espacio de búsqueda
Sintéticas / bench	5 archivos	5	Escenarios controlados de validación

6.2.1. Instancias sintéticas y de *benchmark*

Las instancias de cinco fragmentos constituyen la base cuantitativa de la comparación entre modos de resolución. Su tamaño reducido permite interpretar con claridad las decisiones de selección y ordenación, así como contrastar el comportamiento del baseline temporal frente al modo asistido por LLM.

La instancia `example_instance.json` define un caso base en el que los cinco fragmentos caben dentro del límite temporal ($D_{max} = 45$ s). Comprueba que el

sistema selecciona contenido relevante cuando no es necesario descartar segmentos por restricción de duración.

La variante `example_instance_overlimit.json` reutiliza el mismo contenido textual pero impone un límite más estricto ($D_{max} = 30$ s). Su objetivo es evaluar el recorte de fragmentos cuando no todos pueden incluirse en el resumen.

La instancia `bench_static_vs_dynamic.json` modela un escenario de “salto” narrativo: existe alta coherencia entre dos fragmentos no consecutivos en el orden de entrada, pero baja coherencia con un fragmento puente intermedio. Permite comprobar si el algoritmo evita transiciones débiles al construir la secuencia.

En `bench_irrelevant_middle.json`, el fragmento central simula contenido no educativo (por ejemplo, música o anuncio) intercalado entre segmentos útiles. Se espera que un resumen de calidad omita dicho fragmento y priorice material informativo.

Por último, `bench_disordered.json` contiene los mismos textos que `bench_irrelevant_middle.json`, pero con el array de fragmentos deliberadamente permutado en el JSON. Valida que el sistema no depende del orden de lectura del fichero y que puede reconstruir una secuencia narrativa coherente a partir de puntuaciones semánticas y búsqueda combinatoria.

Estas cinco instancias fueron ejecutadas en la suite experimental principal (`baseline_beam` frente a `llm_beam`) con evaluación post-hoc activada; los resultados correspondientes se analizan en la Sección 7.

6.2.2. Instancias de escalado

Las instancias `dur_*` se generaron a partir de `video_2.json` mediante el script `prepare_mini_instances.py`, conservando progresivamente mayor cantidad de contenido del mismo video. La Tabla 2 recoge sus dimensiones principales.

Cuadro 2: Instancias de escalado derivadas de `video_2.json`.

Instancia	n	Duración origen (s)	D_{max} (s)
<code>dur_5min.json</code>	12	317	79
<code>dur_10min.json</code>	23	612	153
<code>dur_15min.json</code>	34	902	226
<code>dur_20min.json</code>	47	1226	306

A medida que aumenta el número de fragmentos, crece también el coste del precálculo semántico, proporcional al número de pares ordenados de coherencia. Por ello, la evaluación cuantitativa con LLM en el régimen de escalado se completó en el informe para `dur_5min.json`; las instancias de mayor tamaño quedan reservadas para experimentos futuros o para ejecución con caché acumulada, como se comenta en la Sección 8.

6.2.3. Instancias reales y mini videos

Las cuatro instancias `video_*` representan el caso de uso previsto del sistema: resumir material educativo real de duración significativa. Sus tamaños concretos

oscilan entre veinticinco fragmentos (`video_3.json`, unos diez minutos de origen) y cincuenta fragmentos (`video_2.json`, unos veintidós minutos de origen).

Las instancias `mini_video_*` recortan cada video a sus diez primeros fragmentos (aproximadamente cuatro minutos de contenido y D_{max} en torno a sesenta y cinco segundos). Constituyen un punto intermedio entre los *benchmarks* sintéticos y los videos completos: mantienen textos reales, pero con un coste de evaluación LLM aún manejable. Ninguna de estas instancias se incluyó en la suite cuantitativa principal de este informe, aunque comparten formato y pipeline con el resto del corpus.

6.3. Configuración experimental

La evaluación del sistema se realiza mediante un conjunto de experimentos ejecutados sobre distintas instancias del problema. Cada experimento consiste en cargar una instancia, generar un resumen utilizando uno de los modos de resolución disponibles y registrar posteriormente diversas métricas asociadas a la calidad de la solución obtenida.

La ejecución de los experimentos se centraliza en el script `run_experiments.py`, que permite automatizar la comparación entre diferentes configuraciones del sistema. Este componente se encarga de cargar las instancias seleccionadas, ejecutar los algoritmos correspondientes, recopilar las métricas calculadas y almacenar los resultados finales en formato CSV para su posterior análisis.

La implementación contempla dos modos principales de resolución. El primero, denominado `baseline_beam`, utiliza puntuaciones de relevancia uniformes y una medida de coherencia basada únicamente en la proximidad temporal de los fragmentos dentro del video original. Este modo sirve como referencia para evaluar la contribución real de los modelos de lenguaje al proceso de construcción de resúmenes.

El segundo modo, denominado `llm_beam`, incorpora las puntuaciones de relevancia y coherencia calculadas mediante el modelo de lenguaje. En este caso, la búsqueda utiliza directamente la información semántica proporcionada por el LLM para guiar la selección y ordenación de fragmentos. Opcionalmente, este modo puede activar una etapa adicional de refinamiento global mediante evaluación completa de los mejores candidatos generados por el algoritmo de búsqueda.

El algoritmo de exploración utilizado en ambos modos corresponde a una variante de beam search. Durante los experimentos se emplea un ancho de conjunto configurable mediante el parámetro `beam_width`. El valor utilizado por defecto en la implementación es:

`beam_width = 10.`

Este parámetro determina el número máximo de candidatos parciales que se conservan en cada nivel de expansión del espacio de búsqueda. Valores mayores permiten explorar más alternativas, aunque incrementan el coste computacional asociado al proceso de optimización.

La función objetivo utilizada por el algoritmo combina relevancia y coherencia mediante un parámetro de ponderación denominado `coherence_weight`. En los experimentos desarrollados se emplea el valor:

$$\lambda = 0,25.$$

Este valor coincide con el utilizado durante la implementación del solver y refleja una estrategia orientada a priorizar la relevancia de los fragmentos seleccionados sin ignorar completamente la calidad de las transiciones entre ellos.

Cuando se utiliza el modo `llm_beam`, el sistema puede activar una fase adicional de refinamiento global. Para ello se conserva un conjunto de los mejores candidatos producidos por el beam search y posteriormente se evalúan mediante una consulta adicional al modelo de lenguaje. El número de candidatos sometidos a esta evaluación se controla mediante el parámetro `summary_refine_top_m`. El valor por defecto utilizado en la implementación es:

$$\text{summary_refine_top_m} = 3.$$

Esto implica que los tres mejores resúmenes obtenidos según la puntuación local son reevaluados por el modelo de lenguaje antes de seleccionar la solución final.

La infraestructura experimental permite además activar o desactivar la evaluación posterior de las soluciones mediante el argumento `-evaluate`. Cuando esta opción está habilitada, se calculan métricas adicionales de relevancia y coherencia utilizando nuevamente el modelo de lenguaje con el objetivo de analizar la calidad de las soluciones obtenidas.

Finalmente, todos los resultados generados durante la ejecución son almacenados automáticamente en archivos CSV. Cada fila contiene información sobre la instancia evaluada, el solver utilizado, la selección producida y las métricas calculadas. Esta información constituye la base para el análisis comparativo presentado posteriormente en la sección de resultados.

6.4. Métricas

Con el objetivo de analizar el comportamiento de los distintos métodos de resolución, el sistema calcula un conjunto de métricas estructurales y semánticas para cada resumen generado. Estas métricas permiten evaluar tanto el grado de utilización del presupuesto temporal disponible como la calidad de los fragmentos seleccionados y de la secuencia resultante.

Las métricas implementadas se calculan automáticamente al finalizar cada experimento y son almacenadas junto con el resto de los resultados en archivos CSV para su posterior análisis.

6.4.1. Métricas estructurales

Las métricas estructurales describen propiedades cuantitativas de la solución obtenida sin recurrir a evaluaciones semánticas adicionales.

La primera métrica registrada es el número total de fragmentos disponibles en la instancia, denotado por `n_fragments`. Asimismo, se almacena el número de fragmentos finalmente seleccionados para construir el resumen, representado mediante `n_selected`.

También se registra la duración máxima permitida para la instancia (`max_duration`) y la duración total consumida por la solución obtenida (`duration_used`).

A partir de estos valores se calcula el grado de utilización del presupuesto temporal disponible:

$$\text{duration_utilization} = \frac{\text{duration_used}}{\text{max_duration}}.$$

Esta métrica indica qué proporción de la duración máxima permitida ha sido realmente utilizada por el resumen generado.

Otra medida relevante es la cobertura de fragmentos:

$$\text{fragment_coverage} = \frac{\text{n_selected}}{\text{n_fragments}}.$$

Esta cantidad representa la fracción de fragmentos originales que han sido incorporados a la solución final.

Además, se calcula la cobertura temporal respecto al contenido fuente:

$$\text{duration_coverage} = \frac{\text{duration_used}}{\text{source_duration}},$$

donde `source_duration` corresponde a la suma de las duraciones de todos los fragmentos disponibles en la instancia.

Finalmente, se registra la puntuación obtenida por la función objetivo utilizada durante la búsqueda, almacenada mediante la métrica `solver_score`.

6.4.2. Métricas basadas en LLM

Cuando la evaluación semántica está habilitada, el sistema calcula métricas adicionales utilizando nuevamente el modelo de lenguaje.

La primera de ellas es la relevancia media de los fragmentos seleccionados:

$$\text{mean_relevance} = \frac{1}{k} \sum_{i \in S} r_i,$$

donde S representa el conjunto de fragmentos seleccionados y k su cardinalidad.

De forma análoga, se calcula la coherencia media entre fragmentos consecutivos presentes en la solución:

$$\text{mean_coherence} = \frac{1}{k-1} \sum_{j=1}^{k-1} c_{i_j, i_{j+1}}.$$

Esta métrica proporciona una estimación de la calidad de las transiciones producidas dentro del resumen.

A partir de las puntuaciones de relevancia y coherencia también se calcula una medida agregada denominada `objective_score`, que reproduce la función objetivo utilizada durante la evaluación de la solución.

6.4.3. Evaluación global del resumen

Cuando se activa la fase de refinamiento mediante juez macro, el modelo de lenguaje genera una valoración adicional del resumen completo.

Para cada candidato evaluado se obtienen tres puntuaciones normalizadas entre 0 y 1:

- `summary_llm_relevance`: relevancia global del contenido seleccionado.
- `summary_llm_coherence`: coherencia global de la secuencia de fragmentos.
- `summary_llm_score`: valoración global del resumen completo.

Estas puntuaciones son producidas por una consulta específica al modelo de lenguaje en la que se proporciona el texto completo del resumen generado. El modelo devuelve una evaluación global en formato JSON que posteriormente es procesada por el sistema.

Las métricas obtenidas permiten analizar el comportamiento de los distintos métodos de resolución desde perspectivas complementarias. Mientras que las métricas estructurales describen el uso de los recursos disponibles y las características de la selección realizada, las métricas basadas en el modelo de lenguaje ofrecen una estimación semántica de la calidad de los resúmenes producidos.

7. Resultados y análisis

7.1. Casos sintéticos

En esta subsección se analizan los resultados obtenidos sobre las instancias `example_instance.json` y `example_instance_overlimit.json`, descritas en la Sección 6.2. Ambas contienen cinco fragmentos con el mismo contenido textual; difieren únicamente en la restricción de duración ($D_{max} = 45$ s frente a $D_{max} = 30$ s). Los experimentos se ejecutaron comparando los modos `baseline_beam` y `llm_beam` con la configuración descrita en la Sección 6.3.

La Tabla 3 recoge las métricas principales. Para facilitar la comparación entre solvers, la métrica `objective_score` se calculó *post-hoc* mediante el mismo evaluador LLM sobre la selección ya generada (véase la Sección 6.4), de modo que ambos modos puedan contrastarse sobre una escala común.

Cuadro 3: Resultados en instancias sintéticas base (`example_instance` y `example_instance_overlimit`).

Instancia	Solver	k	Util.	Obj.	Selección
<code>example_instance</code>	<code>baseline_beam</code>	5	1,00	1,80	[0,1,2,3,4]
	<code>llm_beam</code>	5	1,00	2,34	[4,2,1,0,3]
<code>example_instance_overlimit</code>	<code>baseline_beam</code>	3	0,90	0,94	[2,3,4]
	<code>llm_beam</code>	3	1,00	1,96	[2,1,4]

En `example_instance`, ambos modos incluyen los cinco fragmentos disponibles y agotan el presupuesto temporal (`duration_utilization` = 1,00). La diferencia aparece en el orden de reproducción: el baseline conserva el orden cronológico del JSON ([0,1,2,3,4]), mientras que `llm_beam` produce la secuencia [4,2,1,0,3]. Esta reordenación eleva la coherencia media entre fragmentos consecutivos de 0,00 a 0,54 y aumenta el `objective_score` de 1,80 a 2,34 (+30 %), pese a mantener la misma relevancia media (0,48). El resultado confirma que, incluso cuando no es necesario descartar fragmentos, las puntuaciones semánticas permiten al beam search construir una secuencia más coherente que la mera proximidad temporal.

En `example_instance_overlimit`, el límite más estricto obliga a seleccionar un subconjunto de tres fragmentos. El baseline elige [2,3,4] —los tres últimos en orden cronológico— y utiliza el 90 % del presupuesto (27 s de 30 s). `llm_beam` selecciona [2,1,4], reordena respecto al input y consume el 100 % del límite (30 s). La mejora en `objective_score` (0,94 $\rightarrow$ 1,96, +109 %) combina una mayor relevancia media (0,42 $\rightarrow$ 0,75) con coherencia entre pares (0,00 $\rightarrow$ 0,55). Además, el juez macro asigna al resumen de `llm_beam` una puntuación global `summary_llm_score` = 0,75 frente a la ausencia de refinamiento en el baseline.

En conjunto, estos dos casos muestran que el modo asistido por LLM mejora tanto la ordenación (cuando caben todos los fragmentos) como la selección bajo recorte (cuando el límite temporal es más estricto), utilizando de forma más eficiente el presupuesto disponible.

7.2. Casos benchmark

Las instancias de *benchmark* evalúan comportamientos semánticos que no pueden resolverse únicamente con duraciones y proximidad temporal. Se analizan `bench_static_vs_dynamic.json`, `bench_irrelevant_middle.json` y `bench_disordered.json`. La Tabla 4 resume los resultados; la Figura 3 ofrece una visión comparativa del `objective_score` post-hoc en las cinco instancias de la suite (incluidas las sintéticas base).

Cuadro 4: Resultados en instancias de *benchmark*.

Instancia	Solver	k	Util.	Obj.	Coh.	Selección
<code>bench_static_vs_dynamic</code>	<code>baseline_beam</code>	2	1,00	1,03	0,80	[0,1]
	<code>llm_beam</code>	2	1,00	1,55	0,80	[0,2]
<code>bench_irrelevant_middle</code>	<code>baseline_beam</code>	1	0,74	0,60	0,00	[0]
	<code>llm_beam</code>	1	0,93	0,68	0,00	[1]
<code>bench_disordered</code>	<code>baseline_beam</code>	4	1,00	2,15	0,37	[1,4,2,3]
	<code>llm_beam</code>	4	1,00	3,18	0,83	[1,4,0,3]

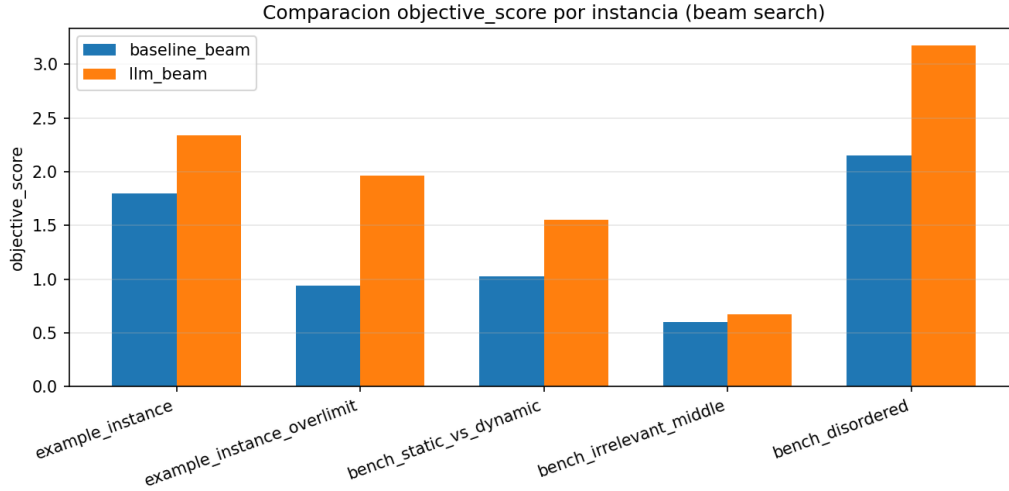


Figura 3: Comparación del `objective_score` post-hoc por instancia y modo de resolución (suite beam).

7.2.1. Salto de coherencia (`bench_static_vs_dynamic`)

Esta instancia modela un escenario en el que la transición directa entre los fragmentos 0 y 2 es semánticamente preferible a la secuencia $0 \rightarrow 1 \rightarrow 2$, porque el fragmento 1 actúa como puente débil. El baseline temporal selecciona $[0,1]$, siguiendo la contigüidad del orden de entrada, con `objective_score` = 1,03. `llm_beam` elige $[0,2]$, evitando el puente intermedio y alcanzando `objective_score` = 1,55. Ambas soluciones utilizan el presupuesto completo de 8 s. El juez macro confirma la calidad del resumen LLM (`summary_llm_score` = 0,85). Este caso demuestra que las puntuaciones de coherencia precalculadas guían correctamente el beam search hacia transiciones globalmente más favorables.

7.2.2. Fragmento irrelevante (`bench_irrelevant_middle`)

Aquí el fragmento central (índice 2) simula contenido no educativo intercalado entre segmentos útiles. Con un límite temporal muy restrictivo ($D_{max} = 10,75$ s), solo cabe un fragmento en el resumen. El baseline selecciona el índice 0 (primer fragmento cronológico), mientras que `llm_beam` elige el índice 1, más relevante según las puntuaciones del LLM. En ambos casos se omiten correctamente los fragmentos que no caben, incluido el irrelevante; la mejora del modo LLM se refleja en una mayor utilización del presupuesto (0,93 frente a 0,74) y en un `objective_score` superior (0,68 frente a 0,60). La evaluación global del juez macro (`summary_llm_score` = 0,85) respalda la selección semántica.

7.2.3. Input permutado (`bench_disordered`)

Este es el contraste más contundente de la suite. Los textos coinciden con `bench_irrelevant_middle`, pero el array de fragmentos aparece permutado en el JSON. El baseline incluye el fragmento irrelevante (índice 2) en la selección $[1,4,2,3]$, obteniendo coherencia media 0,37 y `objective_score` = 2,15. `llm_beam` omite el

fragmento irrelevante y reconstruye la secuencia [1,4,0,3], equivalente a introducción → definición → ejemplo → conclusión. La coherencia media asciende a 0,83 y el `objective_score` a 3,18 (+48 %). La relevancia media mejora de 0,63 a 0,85. Este resultado valida el requisito central del Tema 2: el sistema debe seleccionar y ordenar fragmentos, no limitarse a recortar una subsecuencia del orden de lectura del fichero.

7.2.4. Síntesis de la suite *benchmark*

En las tres instancias de *benchmark*, `llm_beam` supera a `baseline_beam` en `objective_score` post-hoc. La magnitud de la mejora es mayor cuando intervienen simultáneamente selección semántica y reordenación (`bench_disordered`). En los casos con juez macro activo, `llm_beam` obtiene `summary_llm_score` = 0,85 en las tres instancias de *benchmark*, lo que indica coherencia entre la función objetivo local utilizada durante la búsqueda y la valoración global del resumen completo.

7.3. Escalabilidad

Las instancias de escalado permiten estudiar el comportamiento del sistema cuando el número de fragmentos crece respecto a los *benchmarks* de cinco elementos. En esta evaluación se empleó `dur_5min.json`, derivada de `video_2.json` (Sección 6.2), con $n = 12$ fragmentos, duración de origen de 317 s y $D_{max} = 79,24$ s. Los experimentos se ejecutaron con la misma configuración que la suite principal (`beam_width`=10, $\lambda = 0,25$, `summary_refine_top_m`=3, Groq/llama-3.3-70b-versatile) y los resultados se registraron en `results/unified_scaling.csv`.

La Tabla 5 resume las métricas obtenidas. En ambos modos el resumen incluye tres fragmentos ($k = 3$, `fragment_coverage` = 0,25), es decir, una cuarta parte del material disponible. El baseline temporal selecciona los índices [7, 8, 9], una ventana contigua en el orden de entrada; `llm_beam` elige [8, 9, 10], desplazando la ventana un fragmento hacia adelante. Ambas soluciones concentran el resumen en la parte final del material acumulado, coherente con la estrategia del baseline de favorecer proximidad temporal.

Cuadro 5: Resultados de escalado en `dur_5min.json` ($n = 12$).

Solver	k	Util.	Solver score	Obj. score	Coh. media	Juez macro	Selección
baseline_beam	3	0,96	3,21	2,23	0,85	—	[7,8,9]
llm_beam	3	0,99	2,85	2,25	0,90	0,75	[8,9,10]

Desde el punto de vista estructural, `llm_beam` utiliza el 99,0 % del presupuesto temporal (78,44 s de 79,24 s) frente al 96,1 % del baseline (76,16 s). La relevancia media de los fragmentos seleccionados coincide en ambos casos (0,80), mientras que la coherencia media mejora ligeramente con el LLM (0,85 → 0,90). Medido con el evaluador post-hoc común, el `objective_score` asciende de 2,23 a 2,25 (+0,9 %), una diferencia modesta respecto a la suite de cinco fragmentos. Cabe señalar que el

`solver_score` local del baseline (3,21) supera al de `llm_beam` (2,85), porque aquel emplea coherencia temporal durante la búsqueda, mientras que la comparación post-hoc utiliza las puntuaciones semánticas del LLM sobre la selección ya generada.

El juez macro asigna a la solución de `llm_beam` una puntuación global `summary_llm_score` = 0,75 (`summary_llm_relevance` = 0,80, `summary_llm_coherence` = 0,70), coherente con la valoración obtenida en otros escenarios del informe.

El precálculo semántico sobre doce fragmentos requiere evaluar la relevancia de cada fragmento y la coherencia de todos los pares ordenados ($n + n(n - 1) = 144$ consultas en la fórmula de precálculo), más tres evaluaciones del juez macro. En la ejecución se realizaron 147 llamadas nuevas a la API de Groq, con una duración de pared de aproximadamente 6,3 min, partiendo de una caché parcialmente poblada por experimentos previos.

Existen instancias adicionales (`dur_10min.json` con $n = 23$, `dur_15min.json` con $n = 34$ y `dur_20min.json` con $n = 47$), descritas en la Tabla 2, pero no se incluyen resultados cuantitativos para ellas en este informe. Un intento de evaluación sobre `dur_10min.json` fue interrumpido manualmente tras unos 50 min de ejecución, habiendo completado aproximadamente el 46 % del precálculo de coherencia (~ 185 llamadas API adicionales sobre las 524 entradas acumuladas en caché); no se generó CSV ni resultados de `llm_beam` para esa instancia. El crecimiento cuadrático del número de pares de coherencia ($n(n - 1)$) y los retrasos de cortesía, la persistencia de caché en disco y los posibles *rate limits* del proveedor explican que el tiempo real de pared crezca de forma mucho más rápida que en instancias con $n \leq 12$. Por ello, la presente sección se limita a un único punto de la curva de escalado; las implicaciones de este alcance se desarrollan en la Sección 8.

7.4. Discusión

Los resultados de las Secciones 7.1 a 7.3 permiten extraer varias conclusiones transversales sobre el comportamiento del sistema desarrollado, sin repetir el detalle tabular de cada instancia.

En las cinco instancias de la suite principal (`example_instance`, `example_instance_overlimit` y los tres *benchmarks*), el modo `llm_beam` supera a `baseline_beam` en `objective_score` post-hoc en todos los casos. Las mejoras relativas van desde incrementos moderados en escenarios de recorte simple (`bench_irrelevant_middle`: 0,60 $\rightarrow$ 0,68) hasta aumentos más pronunciados cuando intervienen simultáneamente selección semántica y reordenación (`bench_disordered`: 2,15 $\rightarrow$ 3,18; `example_instance_overlimit`: 0,94 $\rightarrow$ 1,96). Este patrón confirma que las puntuaciones de relevancia y coherencia precalculadas aportan información que la proximidad temporal no captura, especialmente cuando el orden del fichero de entrada no refleja un orden narrativo deseable.

El baseline tiende a conservar subsecuencias contiguas del orden de lectura o, en su defecto, los fragmentos finales cronológicos (`example_instance_overlimit`: [2,3,4]; `dur_5min`: [7,8,9]). `llm_beam`, en cambio, puede omitir fragmentos irrelevantes (`bench_disordered`), evitar puentes débiles (`bench_static_vs_dynamic`: [0,2] en lugar de [0,1]), reordenar el contenido (`example_instance`: [4,2,1,0,3]) y aprove-

char mejor el límite de duración cuando el recorte es necesario (`example_instance_overlimit`: utilización 1,00 frente a 0,90).

Cuando está activa la fase de refinamiento, el juez macro proporciona una segunda valoración sobre el resumen completo. En los tres *benchmarks*, `llm_beam` obtiene `summary_llm_score` = 0,85. En `example_instance_overlimit` y `dur_5min` la puntuación global es 0,75; en `example_instance` es 0,70. En conjunto, las valoraciones globales respaldan las decisiones tomadas por el beam search asistido por LLM, aunque en algún caso (`example_instance`) la puntuación local post-hoc ya era alta y el juez macro resulta más conservador. La divergencia ocasional entre `solver_score` (función objetivo empleada durante la búsqueda) y `objective_score` post-hoc (evaluación homogénea con puntuaciones LLM) ilustra que la métrica de búsqueda y la de evaluación final no coinciden necesariamente; el refinamiento global mitiga parcialmente esta discrepancia al elegir entre los candidatos del conjunto.

`Baseline_beam` cumple una función metodológica clara: aísla el efecto de incorporar evaluación semántica, manteniendo idéntico algoritmo de búsqueda y mismos parámetros (`beam_width`, λ). Sus limitaciones —coherencia nula o baja cuando el orden cronológico no es narrativamente adecuado, inclusión de fragmentos irrelevantes cuando el input está permutado— hacen visible el valor añadido del LLM sin necesidad de comparar con métodos externos.

La suite de cinco fragmentos permite interpretar con detalle cada decisión de selección y ordenación, pero no representa el régimen de tamaño previsto para videos educativos completos (25–50 fragmentos en `video_*`). El único punto de escalado evaluado (`dur_5min`, $n = 12$) muestra una ventaja modesta del LLM en `objective_score` (+0,9 %) con selecciones muy similares (ventanas contiguas desplazadas). No es posible, con los datos disponibles, trazar una curva tiempo–calidad versus n ni afirmar cómo se comportaría el sistema en `dur_20min` ($n = 47$). La interrupción del experimento sobre `dur_10min` evidencia además que el coste del precálculo semántico puede convertirse en el cuello de botella principal antes que el propio beam search.

El pipeline precálculo LLM $\rightarrow$ beam search $\rightarrow$ juez macro funciona de extremo a extremo y mejora la calidad semántica medida post-hoc respecto al baseline temporal en todos los escenarios evaluados cuantitativamente. La ganancia es mayor cuando el problema exige razonamiento sobre coherencia no adyacente, filtrado de ruido o reconstrucción de orden narrativo. En régimen de escalado, los resultados son preliminares y están condicionados por el coste cuadrático de la matriz de coherencia; completar la evaluación sobre instancias de mayor tamaño constituye trabajo pendiente, como se detalla en las secciones siguientes.

8. Limitaciones y mejoras futuras

A pesar de los resultados positivos descritos en la Sección 7, la propuesta presenta limitaciones metodológicas, computacionales y de alcance que conviene explicitar para contextualizar correctamente las conclusiones del trabajo.

8.1. Limitaciones

La evaluación semántica del sistema descansa por completo en consultas a un LLM alojado en un servicio externo. En los experimentos documentados se empleó Groq con el modelo `llama-3.3-70b-versatile` y temperatura cero, con el fin de favorecer la reproducibilidad (Sección 5.3). No obstante, persisten factores fuera del control del desarrollador: variaciones entre modelos o versiones, límites de cuota (*rate limits*), latencia de red y posibles interrupciones del servicio. Un intento de escalado sobre `dur_10min.json` fue detenido tras unos 50 min de ejecución, habiendo consumido aproximadamente 185 llamadas API adicionales sin completar el precálculo.

El diseño adoptado calcula una puntuación de coherencia para cada par ordenado de fragmentos, lo que implica $n(n-1)$ consultas además de las n de relevancia. Para $n = 12$ (`dur_5min`) el precálculo requirió 147 llamadas nuevas y unos 6,3 min de pared; para $n = 23$ (`dur_10min`) la fórmula de precálculo en frío ascendería a 552 consultas solo en la fase de puntuaciones, sin contar el juez macro ni la evaluación post-hoc. El crecimiento cuadrático convierte el precálculo semántico en el principal cuello de botella del sistema para instancias de tamaño moderado, antes incluso que el propio beam search, cuyo coste combinatorio queda acotado por el parámetro `beam_width`. Factores de implementación —reescritura completa del fichero de caché en cada respuesta nueva, retrasos de cortesía entre llamadas y esperas ante HTTP 429— agravan aún más el tiempo real de ejecución respecto a estimaciones lineales basadas únicamente en la latencia de la API.

La suite cuantitativa principal se limitó a cinco instancias de cinco fragmentos y a un único punto de escalado (`dur_5min`, $n = 12$). Las instancias `dur_10min`, `dur_15min` y `dur_20min` están generadas y documentadas (Tabla 2), pero carecen de resultados en este informe. Los cuatro videos educativos completos (`video_1.json`–`video_4.json`, entre 25 y 50 fragmentos) y sus versiones recortadas `mini_video_*` no formaron parte de la evaluación con LLM, precisamente por el coste asociado a la matriz de coherencia.

El beam search empleado (`beam_width=10`) explora de forma heurística el espacio de secuencias válidas y no garantiza la optimalidad global de la solución. Podrían existir combinaciones no alcanzadas por el conjunto que obtengan mayor `solver_score`. Asimismo, la función objetivo pondera relevancia y coherencia pairwise con un λ fijo (0,25), sin adaptación al tipo de instancia ni validación sistemática de sensibilidad de ese hiperparámetro.

Los fragmentos de entrada se obtienen agrupando subtítulos consecutivos hasta alcanzar aproximadamente veinte a veinticinco segundos de duración acumulada (Sección 4.2). Esta regla fija puede dividir ideas a mitad de exposición o fusionar temas distintos en un mismo segmento, condicionando tanto las puntuaciones del LLM como las decisiones del solver. No se ha evaluado el impacto de otras estrategias de segmentación ni la cobertura temática del resumen respecto al video completo.

Todas las métricas semánticas del informe provienen de evaluaciones automáticas mediante LLM. No se realizó evaluación humana (juicios de expertos o usuarios), ni se emplearon métricas clásicas de resumen extractivo como ROUGE o BERTScore, que habrían permitido contrastar la salida con referencias textuales. La calidad reportada refleja, por tanto, criterios definidos en los *prompts* del sistema, no un

juicio independiente sobre la utilidad educativa del resumen.

8.2. Mejoras futuras

Varias líneas de trabajo pueden abordar las limitaciones anteriores sin modificar la arquitectura general del pipeline.

Completar la evaluación de escalado reanudando el experimento sobre `dur_10min.json` —la caché parcial acumulada permite continuar sin repetir consultas ya realizadas— y, progresivamente, las instancias `dur_15min` y `dur_20min`. Paralelamente, mejorar la persistencia de la caché y revisar la política de retrasos entre peticiones reduciría el tiempo de pared observado en instancias con $n \geq 23$. Para tamaños mayores, explorar matrices de coherencia dispersas —evaluando únicamente pares prometedores según proximidad temporal o similitud léxica— podría mantener la calidad semántica con un coste subcuadrático.

Incorporar evaluación humana sobre un subconjunto de resúmenes generados, complementada con métricas automáticas independientes del LLM utilizado en el solver, fortalecería la validez externa de los resultados. Ajustar la estrategia de fragmentación o permitir que el límite D_{max} se configure por instancia —en lugar de fijarlo siempre al 25 % de la duración total— ampliaría la flexibilidad del sistema en escenarios de uso distintos al experimentado en este trabajo.

9. Conclusiones

El presente trabajo abordó el Tema 2 del Proyecto Final de Inteligencia Artificial: construir resúmenes de videos educativos como un problema de selección y ordenación óptima de fragmentos bajo una restricción de duración, incorporando un modelo de lenguaje de gran tamaño como componente funcional del proceso. A continuación se responden de forma explícita las preguntas que orientan la evaluación del proyecto.

9.1. ¿Se resolvió el problema?

Sí. Se diseñó, implementó y evaluó un pipeline completo que cumple los requisitos formulados en las Secciones 2 y 3. El sistema carga instancias de fragmentos con transcripción y límite temporal; precalcula puntuaciones de relevancia y coherencia mediante un LLM; construye candidatos de resumen mediante beam search respetando D_{max} ; y selecciona la solución final mediante un juez macro que evalúa el texto concatenado del resumen. La arquitectura separa de forma clara la evaluación semántica (LLM) de la optimización combinatoria (beam search), de acuerdo con el enfoque descrito en la Sección 4. Los experimentos sobre cinco instancias controladas y un caso de escalado demuestran que el flujo se ejecuta de extremo a extremo y produce resúmenes válidos en todos los escenarios probados.

9.2. ¿Qué aportó el uso del LLM?

El LLM no genera directamente el resumen, sino que aporta las puntuaciones semánticas que guían la búsqueda: relevancia por fragmento, coherencia entre pares y, en una fase posterior, valoración global de los mejores candidatos. Frente al `baseline_beam`, que utiliza relevancia uniforme y coherencia basada en proximidad temporal, el modo `llm_beam` mejoró el `objective_score` post-hoc en las cinco instancias de la suite principal. Las ganancias fueron especialmente marcadas cuando el problema exigía filtrar contenido irrelevante, evitar transiciones débiles o reconstruir un orden narrativo a partir de un input permutado (`bench_disordered`). En escenarios más sencillos o en el único punto de escalado evaluado (`dur_5min`), las diferencias fueron menores, lo que indica que el valor del LLM depende del grado en que la mera contigüidad temporal resulta insuficiente. El juez macro aportó, además, una segunda opinión coherente con las decisiones del solver en la mayoría de los casos (`summary_llm_score` entre 0,70 y 0,85).

9.3. ¿Qué harías en una versión futura?

Priorizaría tres direcciones, alineadas con las limitaciones identificadas en la Sección 8. En primer lugar, completar la evaluación de escalado sobre las instancias `dur_*` y, progresivamente, sobre videos reales (`mini_video_*` y `video_*`), optimizando el precálculo de coherencia para que el coste no impida evaluar el caso de uso previsto. En segundo lugar, reducir el tiempo y la fragilidad de los experimentos largos mediante una caché más eficiente y, si fuera necesario, estrategias de reducción del número de pares evaluados. En tercer lugar, complementar las métricas basadas en LLM con evaluación humana y, opcionalmente, con estudios de ablation sobre `beam_width` y sobre el modelo evaluador, para separar con mayor rigor la calidad del algoritmo de la del proveedor de inferencia.

En conjunto, el trabajo demuestra que integrar un LLM como evaluador semántico dentro de un esquema de optimización combinatoria es viable y mejora la calidad de los resúmenes en escenarios donde importa tanto la selección como el orden de los fragmentos. La principal barrera para un despliegue práctico sobre videos completos no es el algoritmo de búsqueda, sino el coste cuadrático del precálculo de coherencia; superar esa barrera constituye el paso natural hacia una versión del sistema preparada para material educativo de longitud real.

A. Configuraciones experimentales

La Tabla 6 recoge los parámetros empleados en los experimentos documentados en las Secciones 6.3 y 7. Estos valores coinciden con los *defaults* de `run_experiments.py` salvo donde se indica lo contrario.

Cuadro 6: Parámetros experimentales utilizados en la evaluación.

Parámetro	Valor
Modos de resolución	baseline_beam, llm_beam
beam_width	10
coherence_weight (λ)	0,25
summary_refine_top_m	3
Proveedor LLM	Groq
Modelo	llama-3.3-70b-versatile
Temperatura	0
Flags de ejecución	-llm -evaluate
Script de entrada	src/run_experiments.py
Salida CSV (suite bench)	results/experiments_bench_beam.csv
Salida CSV (escalado)	results/unified_scaling.csv
Notas automáticas	informe/notas_experimentos.md
Caché LLM	.llm_cache.json (persistente)

En cada ejecución con `-llm`, el script corre siempre `baseline_beam` y, adicionalmente, `llm_beam`, produciendo dos filas por instancia. La opción `-evaluate` activa el cálculo post-hoc de `mean_relevance`, `mean_coherence` y `objective_score` mediante el evaluador LLM, así como las métricas del juez macro cuando corresponde.

B. Prompts utilizados

Los siguientes listados reproducen literalmente las plantillas definidas en `src/llm/prompts.py`. Las cadenas `{fragment_text}`, `{first_fragment}`, `{second_fragment}`, `{summary_text}` y `{max_duration}` se sustituyen en tiempo de ejecución por los textos y valores de la instancia.

B.1. Relevancia de fragmento (fragment_prompt)

Evalúa la relevancia de este fragmento para un resumen educativo.
Devuelve un valor numérico entre 0.0 y 1.0.

Fragmento:

`{fragment_text}`

Respuesta esperada: un solo número.

B.2. Coherencia entre pares (coherence_prompt)

Evalúa la coherencia entre dos fragmentos consecutivos.
Devuelve un valor numérico entre 0.0 y 1.0.

Primer fragmento:

`{first_fragment}`

Segundo fragmento:

`{second_fragment}`

Respuesta esperada: un solo número.

B.3. Evaluación global del resumen (summary_evaluation_prompt)

Evalúa el siguiente resumen extractivo de un video educativo.

Considera si los fragmentos seleccionados representan bien el contenido y si el orden es coherente. La duración máxima permitida es {max_duration} segundos.

Resumen:

{summary_text}

Devuelve un objeto JSON con tres claves numéricas entre 0.0 y 1.0:

- "relevance": qué tan relevante es el conjunto para un resumen educativo
- "coherence": qué tan coherente es la secuencia de fragmentos
- "overall": calidad global del resumen

Respuesta esperada: solo el JSON, sin texto adicional.

Referencias

- [1] Docencia de Inteligencia Artificial. Orientación del proyecto final inteligencia artificial 2025–2026. Material docente del curso, 2025. Tema 2: Construcción de resúmenes como selección óptima de segmentos.
- [2] Groq Inc. Groq cloud documentation, 2024.
- [3] Meta AI. Llama 3.3 model card, 2024.
- [4] Ani Nenkova and Kathleen McKeown. Automatic summarization. In *Foundations and Trends in Information Retrieval*, volume 5, pages 103–233. Now Publishers, 2011.
- [5] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020.